OBJECT ORIENTED PROGRAMMING WITH C++

ASSIGNMENT 1 – Namma Metro Smart-Card Fare System

CO1 | Tasks 1–12 | Total: 50 Marks

Student detail	To be completed
Name	HARIPRASAD R
Register Number	192421179
Department / Year	B.TECH IT – 3rd Year
Submission Date	10-11-2026
GitHub Repository	https://github.com/<username>/DSA01-<RegisterNumber>-Assignment1
Compiler / Version	g++ Dev c++ (record your installed version)

Important submission note: This report is a worked answer/template. You must create the GitHub repository, compile and execute each source file on your own computer, capture your own screenshots, record your actual compiler version and timings, and commit each task progressively. Do not claim that the sample measurements or outputs below were collected on your laptop.

Assumptions used throughout

- Base fares are Rs 10, Rs 20 and Rs 30 for Inner, Middle and Outer zones.
- Peak hours are [08:00, 10:00) and [17:00, 20:00): start included, end excluded. Therefore 10:00 AM and 8:00 PM are off-peak.
- The peak surcharge is 20% of the base fare. Fares are represented in integer paise; all supplied fares and percentage calculations are exact in this assignment.
- Student and senior-citizen concessions are mentioned in the scenario but no percentage or combination rule is supplied. The initial implementation therefore exposes a configurable concession percentage and uses 0% until the faculty specifies a value. Do not invent a percentage for submission.
- A journey is accepted only if balance - fare >= Rs 50. The minimum balance is a post-payment reserve, not a minimum initial balance.
- Weekend discount and monthly-pass rules are introduced only in Task 12. For Task 12, weekend discount is 10% of the zone base fare and does not combine with the peak surcharge. Monthly pass covers two selected zones; a valid pass makes the fare zero for journeys in those zones.

Task 1 – Understand the Problem and Identify the Objects (4 marks)

Object-oriented analysis

Object	Data (state)	Behaviour (operations)	Justification
SmartCard	card ID, balance in paise, card status	canTravel(fare), deduct(fare), recharge(amount)	Owns stored value and enforces the Rs 50 reserve invariant. Private balance prevents unrelated code changing it.
Passenger	passenger ID, category (regular/student/senior)	requestJourney(), concession eligibility	Passenger category is needed to select a concession policy; separating it avoids mixing identity with card money.
Journey	origin, destination/zone,	validate(), calculateFare(),	Represents one travel event

	entry time, date, computed fare, status	complete()/reject()	and preserves an auditable record of the inputs and outcome.
Zone / ZoneFare	zone enum, base fare	lookupBaseFare()	A finite set of named zones prevents arbitrary numeric zone values and centralizes fare data.
FareTable / FarePolicy	base fares, surcharge percentage, time windows, concession policy	baseFare(), isPeak(), calculate()	Fare rules change independently of a card; a dedicated policy avoids duplicate fare logic.

Procedural style: keep arrays/variables such as balance[], zoneFare[], passengerType[] and free functions calculateFare(), isPeak() and deductFare(). Each function receives the required data. This can work, but global state and shared rules become harder to protect as the program grows.

OOP style: encapsulate card balance inside SmartCard, represent a journey with Journey, and isolate pricing in FarePolicy/FareTable. Objects collaborate through public methods. A change to the pricing rule is concentrated in the policy rather than scattered across gate code.

Where the four OOP principles appear

- Encapsulation: SmartCard keeps balance private; only validated recharge/deduct methods modify it.
- Abstraction: Gate code asks FarePolicy to calculate a fare and SmartCard whether travel is allowed; it need not know every calculation detail.
- Inheritance: StudentPassenger and SeniorPassenger may derive from Passenger if they have genuinely different passenger behaviour. Alternatively, a PassengerCategory enum and concession policy may be simpler.
- Polymorphism: a base FarePolicy interface can have StandardFarePolicy and WeekendFarePolicy implementations; a Passenger concession strategy can also be selected at runtime.

Journey invariants / correctness rules

1. Zone must be one of the declared valid zones; unknown zone input is rejected.
2. Hour must be 0–23 and minute 0–59.
3. Peak is true only for 08:00–09:59 and 17:00–19:59.
4. Money is non-negative and stored in integer paise.
5. Fare is calculated once using the selected policy; no duplicated magic fare values.
6. A journey is accepted only if balance - fare >= 5000 paise; otherwise neither fare nor balance is changed.
7. After every accepted journey, balance remains at least Rs 50; rejected journeys do not change the balance.
8. All arithmetic is checked for valid input and sufficient numeric range.

Alternative concept and complexity

In a procedural design, free functions receive a CardRecord and fare-rule arrays. Per journey, a constant number of comparisons and arithmetic operations gives O(1) time. Per card, a fixed number of fields gives O(1) space. The asymptotic complexity is the same as OOP. When a fare rule changes, a centralized procedural function may require only one edit; however, if the rule was duplicated in several functions, every copy must be updated. OOP does not automatically reduce complexity, but encapsulation and a single FarePolicy reduce the number of places that need modification.

Task 2 – Tokens, Keywords, Identifiers and Constants (3 marks)

Five token categories with examples from this assignment:

Token type	Examples	Explanation
Keywords	class, public, private, enum, constexpr, if,	Reserved words with defined C++

	else, switch, return	meaning.
Identifiers	SmartCard, balancePaise, calculateFare, PEAK_PERCENT	Programmer-chosen names; cannot be keywords.
Literals / constants	5000, 20, true, 'A', "Fare"	Fixed values written directly in source code.
Operators	+ , -, *, /, %, ==, >=, &&, ?:, =	Perform arithmetic, comparison, logic or assignment.
Punctuators	; , { } ( ) [ ] :: ,	Separate statements and structure declarations/expressions.

Constants design: use constexpr for compile-time values such as base fares, surcharge percent and minimum reserve. constexpr is typed, scoped and checked by the compiler. const is suitable for values known only at runtime but not intended to change. #define performs textual substitution, has no C++ type or normal scope, and can create precedence/name-collision problems.

```
#include <array>
#include <cstdint>

enum class Zone { Inner = 0, Middle = 1, Outer = 2 };

constexpr std::array<long long, 3> BASE_FARE_PAISE{
    1000LL, 2000LL, 3000LL
};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000LL;
constexpr int PEAK_START_MORNING = 8;
constexpr int PEAK_END_MORNING = 10;
constexpr int PEAK_START_EVENING = 17;
constexpr int PEAK_END_EVENING = 20;
```

enum class is safer than an unscoped enum because values are scoped (Zone::Inner), do not implicitly convert to int, and avoid accidental name collisions. Identifier convention: types/classes use PascalCase; functions and variables use lowerCamelCase; constants use UPPER_SNAKE_CASE. Names should be descriptive and should not begin with a digit.

Alternative concept and complexity

With #define, e.g. #define INNER_FARE 1000, the preprocessor substitutes text before compilation. It has no type checking and no normal namespace scope. constexpr provides type checking and scope. Neither necessarily occupies runtime storage when used as a compile-time constant. Runtime cost is normally unchanged (O(1) lookup/arithmetic); the major difference is safety and maintainability, not speed.

Task 3 – Choose Data Types and Check Type Compatibility (4 marks)

Value	Recommended type	Reason / range consideration
Fare and balance	long long paise	Exact integer money; avoids binary floating-point rounding.
Zone	enum class Zone	Only named valid zone values.
Hour / minute	int or unsigned int after validation	Small bounded values; int simplifies input validation.
Peak flag / accepted	bool	Two-state result.
Passenger category	enum class PassengerCategory	Regular, Student, Senior.
Journey counter / daily total count	long long	Safely counts millions and future growth.
Collection total	long long paise	Large integer total; check maximum based on journey count and fare.
Single character	char	Character input, not a numeric fare.
Scientific calculations only	double	Not preferred for currency; use only where approximation is acceptable.
Inferred local value	auto	Useful when initializer clearly determines the intended type; not a replacement for

		deliberate money types.

Money should be stored as integer paise: Rs 10.20 becomes 1020 paise. Addition/subtraction and percentage calculations can be performed with integer arithmetic. For the supplied 20% surcharge, base * 120 / 100 is exact because each base fare is a multiple of 100 paise.

Type compatibility: in an expression such as int + double, the int is converted to double (usual arithmetic conversions). A bool used numerically converts to 0 or 1. A char is generally promoted to int in arithmetic. Signed/unsigned comparisons can convert the signed operand to unsigned, producing surprising results for negative values.

Use sizeof(T) on the target compiler to report actual sizes. The following common Windows 64-bit g++ model is illustrative; confirm it locally:

Type	Typical sizeof on x86-64 MinGW	Relevant range / note
bool	1 byte	false/true
char	1 byte	at least -127..127 or 0..255 depending on signedness
int	4 bytes	INT_MIN..INT_MAX; commonly -2,147,483,648..2,147,483,647
unsigned int	4 bytes	0..4,294,967,295
long long	8 bytes	at least -9.22e18..+9.22e18
float	4 bytes	about 6–7 decimal digits precision
double	8 bytes	about 15–16 decimal digits precision

For 3,000,000 journeys at the maximum original fare with 20% surcharge, maximum single fare is Rs 36 = 3,600 paise. Total is 10,800,000,000 paise (Rs 108,000,000), greater than INT_MAX (2,147,483,647). A 32-bit int is insufficient for the total; long long is suitable for this scale.

Alternative concept: double rupees

A double solution stores values such as 10.00 and adds them in floating point. Each addition is O(1), and a double is typically 8 bytes, same as a long long on the target platform. However, many decimal fractions cannot be represented exactly in binary. Repeated additions can accumulate a small error. The exact observed error depends on values, order and compiler; run a loop adding the fare n times and compare with integer paise converted to rupees. Integer paise is preferred because it makes currency arithmetic exact within the chosen range.

Task 4 – Control Implicit Conversions (4 marks)

Conversion points to identify:

- int division: a / b with both operands int is evaluated as integer division before assignment to double. 7 / 2 becomes 3, then 3 converts to 3.0.
- Explicit floating division: static_cast<double>(a) / b converts a to double first, producing 3.5.
- Character arithmetic: 'A' + 2 promotes char to int, adds 2, then assignment converts the result back to char ('C' on ASCII-compatible systems). Printing ch + 1 promotes ch to int and prints its numeric code.
- Boolean initialization: int 5 converts to bool true; output is 1 by default and true under boolalpha.
- Signed/unsigned comparison: with u unsigned int and s = -1, s is converted to unsigned. On common 32-bit unsigned int, -1 becomes UINT_MAX, so s < u is false and output is s >= u. -Wsign-compare typically warns.
- Narrowing: int x{3.7}; is ill-formed because list initialization rejects narrowing; int x = 3.7; is allowed with a warning or diagnostic depending on flags and loses the fraction.
- static_cast should be used when a deliberate numeric conversion is needed, for example static_cast<double>(a) / b.

Prediction – provided program

Write this prediction in the assignment before running the program. The expected output on a typical ASCII-compatible g++ system is:

```
3 3.5
C 68
1 true
s >= u
```

Line-by-line explanation:

Output line	Rule applied
3 3.5	a / b is integer division (7/2 = 3); static_cast<double>(a) makes the second division floating point.
C 68	'A' + 2 uses integral promotion and gives the character C when converted back to char; ch + 1 promotes C to its integer code 68.
1 true	int 5 converts to bool true; bool prints as 1 normally and as true after std::boolalpha.
s >= u	The negative signed int is converted to unsigned for the comparison; its unsigned value is very large.

Corrected explicit-conversion style:

```
#include <iostream>
int main() {
    int a = 7, b = 2;
    double c = static_cast<double>(a / b); // still 3.0: cast occurs after division
    double d = static_cast<double>(a) / static_cast<double>(b);
    char ch = static_cast<char>('A' + 2);
    bool flag = (5 != 0);
    std::cout << c << ' ' << d << '\n';
    std::cout << ch << ' ' << static_cast<int>(ch) + 1 << '\n';
    std::cout << flag << ' ' << std::boolalpha << flag << '\n';
}
```

Alternative concept and complexity

Explicit casts do not make arithmetic faster or slower in a meaningful way; both approaches are O(1) and use the same storage for the same types. Explicit conversions make intent visible. Brace initialization catches narrowing during compilation. It does not prevent every logical error (such as signed/unsigned comparison), so validation and suitable types are still required.

Task 5 – Correct Expressions with the Right Precedence (4 marks)

```
bool isPeak(int hour) {
    return (hour >= 8 && hour < 10) ||
           (hour >= 17 && hour < 20);
}
long long applyPeakSurcharge(long long basePaise, bool peak) {
    return peak ? basePaise * (100 + PEAK_SURCHARGE_PERCENT) / 100
                : basePaise;
}
long long applyConcession(long long farePaise, int concessionPercent) {
    return farePaise * (100 - concessionPercent) / 100;
}
```

The peak expression uses relational operators before logical AND, and logical AND before logical OR. Parentheses make the two time windows explicit. The conditional operator ?: has lower precedence than arithmetic; parentheses or a separate function improve clarity. Compound assignment such as balance -= fare is equivalent in effect to balance = balance - fare, with the left operand evaluated once.

x++ yields the old value and increments afterward; ++x increments first and yields the new value. Avoid i = i++: in modern C++ the expression has well-defined sequencing rules in many contexts, but it is confusing and historically problematic; simply write ++i or i += 1.

Integer division truncates toward zero. Thus 10/3 = 3, -10/3 = -3, and -10%3 = -1. The remainder has the sign of the left operand under C++ truncation-toward-zero division.

Prediction – provided program

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Operator-by-operator trace:

Line	Evaluation
11 16	* before +: 5 + (3*2)=11. Parentheses force (5+3)*2=16.
9 6 4	x++ yields old x=5 then x becomes 6; ++y makes y=4 and yields 4; sum z=9.
6 4	x > y compares 6 and 4, so ?: selects 6. % and * have same precedence and associate left-to-right: (6 % 4)*2 = 4.
3 1 -3 -1	Integer division truncates toward zero; remainder satisfies dividend = quotient*divisor + remainder.
18	for loop adds i values 0, 3, 6, 9: total 18.

Alternative: peak lookup table

```
constexpr bool PEAK_HOUR[24] = {
    false,false,false,false,false,false,false,false,
    true,true,false,false,false,false,false,false,
    false,true,true,true,false,false,false,false
};
// Use PEAK_HOUR[hour] only after validating 0 <= hour && hour < 24.
```

Both the expression and table lookup are O(1). A bool table needs 24 bool elements (commonly 24 bytes, but sizeof(bool) should be checked). It may reduce repeated comparisons at the cost of extra data and a maintenance risk if the table is entered incorrectly. A single isPeak() function with explicit boundaries is easier to review for this small rule.

Task 6 – Design the Control Structures (4 marks)

Use switch for selecting one of a small set of named zones; if/else for validation and balance rules; ?: for a compact value selection only when readable. A counted for loop is suitable for a known array of journeys; range-based for is safest for iterating all trips; while is useful when reading until a sentinel or end-of-file. Use continue to skip invalid input and break to stop on a fatal condition. A switch with no matching case and no default simply executes no case; include default to reject unknown values.

SIMATS-style pseudocode

```
ALGORITHM ProcessJourneys
INPUT: journeys, startingBalancePaise
CONSTANT: minimumBalancePaise = 5000
FOR each trip IN journeys
    IF trip.zone is invalid OR trip.hour < 0 OR trip.hour > 23
        DISPLAY "Invalid journey"
        CONTINUE
    END IF
    CASE trip.zone OF
        Inner: baseFare <- 1000
        Middle: baseFare <- 2000
        Outer: baseFare <- 3000
```

```
        OTHERWISE: DISPLAY "Unknown zone"; CONTINUE
    END CASE
    peak <- IsPeak(trip.hour)
    IF peak THEN
        fare <- baseFare * 120 / 100
    ELSE
        fare <- baseFare
    END IF
    IF balance - fare >= minimumBalancePaise THEN
        balance <- balance - fare
        DISPLAY fare and balance
    ELSE
        DISPLAY "Denied (low balance)"
    END IF
END FOR
OUTPUT balance
END ALGORITHM
```

Completed C++ program (TODO 1 and TODO 2)

```
#include <iomanip>
#include <iostream>

enum class Zone { Inner, Middle, Outer };
constexpr int BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

bool isPeak(int hour) {
    return (hour >= 8 && hour < 10) ||
           (hour >= 17 && hour < 20);
}

long long fare(Zone z, bool peak) {
    long long base = 0;
    switch (z) {
        case Zone::Inner:  base = BASE_PAISE[0]; break;
        case Zone::Middle: base = BASE_PAISE[1]; break;
        case Zone::Outer:  base = BASE_PAISE[2]; break;
        default: return -1; // defensive fallback
    }
    if (peak)
        base = base * (100 + PEAK_SURCHARGE_PERCENT) / 100;
    return base;
}

bool canTravel(long long balance, long long f) {
    return f >= 0 && balance >= f &&
           balance - f >= MIN_BALANCE_PAISE;
}

int main() {
    long long balance = 10000;
    struct Trip { Zone zone; bool peak; };
    Trip trips[] = {
        {Zone::Inner, false}, {Zone::Outer, true},
        {Zone::Middle, false}
    };
    for (const Trip& t : trips) {
        long long f = fare(t.zone, t.peak);
        if (canTravel(balance, f)) {
            balance -= f;
            std::cout << "Fare Rs " << f / 100 << '.'
                      << std::setw(2) << std::setfill('0')
```

```
                        << f % 100 << std::setfill(' ');
            } else {
                std::cout << "Denied (low balance)";
            }
            std::cout << "  balance Rs " << balance / 100 << '\n';
        }
    }
```

Expected output:

```
Fare Rs 10.00  balance Rs 90
Fare Rs 36.00  balance Rs 54
Denied (low balance)  balance Rs 54
```

Boundary test: 10:00 AM is off-peak because hour 10 fails hour < 10. 8:00 PM is off-peak because hour 20 fails hour < 20. For minute-accurate handling, store minutes since midnight and use [480,600) and [1020,1200).

Alternative: array instead of switch

An array lookup and switch are both O(1). The array occupies three long long elements (typically 24 bytes); the switch may compile to branches or a jump table. A fourth zone requires adding a fare entry and a new enum value; a switch needs a new case. A table indexed by enum needs bounds validation and consistent enum-to-index mapping.

Task 7 – From Procedural to Object-Oriented (4 marks)

Procedural version

```
#include <iostream>
constexpr long long MIN_RESERVE = 5000;
bool canTravel(long long balance, long long fare) {
    return fare >= 0 && balance >= fare &&
           balance - fare >= MIN_RESERVE;
}
bool deduct(long long& balance, long long fare) {
    if (!canTravel(balance, fare)) return false;
    balance -= fare;
    return true;
}
int main() {
    long long balance = 10000;
    long long f = 2000;
    if (deduct(balance, f))
        std::cout << "Accepted; balance paise=" << balance << '\n';
    else
        std::cout << "Rejected\n";
}
```

Object-oriented version

```
#include <iostream>
class SmartCard {
private:
    long long balancePaise;
    static constexpr long long MIN_RESERVE_PAISE = 5000;
public:
    explicit SmartCard(long long initial) : balancePaise(initial) {
        if (initial < 0) balancePaise = 0;
    }
    long long balance() const { return balancePaise; }
    bool canTravel(long long farePaise) const {
        return farePaise >= 0 && balancePaise >= farePaise &&
               balancePaise - farePaise >= MIN_RESERVE_PAISE;
    }
    bool deduct(long long farePaise) {
```

```
        if (!canTravel(farePaise)) return false;
        balancePaise -= farePaise;
        return true;
    }
};
int main() {
    SmartCard card(10000);
    if (card.deduct(2000))
        std::cout << "Accepted; balance paise=" << card.balance() << '\n';
    else
        std::cout << "Rejected\n";
}
```

Both approaches are O(1) per fare check/deduction and use O(1) memory per card. In the procedural version, every function receiving a mutable balance must follow the rule. In the class version, private data plus validated methods make it harder for unrelated code to create a negative balance or violate the reserve. Changing the reserve is one constant edit in either centralized design.

Alternative: public balance

Making balance public does not change time or space complexity. It does weaken the invariant: any code can write card.balancePaise = -100 or subtract without checking the Rs 50 reserve. Private state and controlled methods are therefore recommended.

Task 8 – Read and Print Data Safely (4 marks)

With cin >> value, check if (!cin). On failure, call cin.clear() to reset failbit, then cin.ignore(numeric_limits<streamsize>::max(), '\n') to discard the invalid line. Use fixed and setprecision(2) for rupee display. setw affects the next field only; setfill remains active until changed.

```
#include <iomanip>
#include <iostream>
#include <limits>

int main() {
    int hour{};
    std::cout << "Enter hour (0-23): ";
    if (!(std::cin >> hour)) {
        std::cout << "Invalid numeric input\n";
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        return 1;
    }
    if (hour < 0 || hour > 23) {
        std::cout << "Hour out of range\n";
        return 1;
    }
    long long paise = 1234;
    std::cout << "Fare Rs " << std::fixed << std::setprecision(2)
              << static_cast<double>(paise) / 100.0 << '\n';
}
```

Alternative getline + stoi: read a complete line into std::string, call stoi inside try/catch, and verify that the entire string was consumed (use the position argument). Reading/conversion is O(L) for a line of length L and uses O(L) temporary string space. cin >> int also processes characters in O(L), but leaves the invalid token in the stream until it is cleared and discarded. getline gives clearer line-level recovery and can reject trailing junk such as '12abc' when full consumption is checked.

Task 9 – Implement and Test the Fare Logic (4 marks)

Core validation test matrix (fares in rupees):

Case	Zone	Time	Expected base / rule	Expected fare
Inner off-peak	Inner	07:59	10	10.00
Inner peak	Inner	08:00	10 × 1.20	12.00
Middle peak	Middle	09:59	20 × 1.20	24.00
Middle boundary	Middle	10:00	off-peak	20.00
Outer peak	Outer	17:00	30 × 1.20	36.00
Outer boundary	Outer	20:00	off-peak	30.00
Outer off-peak	Outer	12:00	30	30.00
Invalid zone	Unknown	12:00	reject input	No fare
Invalid hour	Inner	24:00	reject input	No fare
Invalid recharge	—	—	negative recharge rejected	No balance change
Non-numeric hour	Inner	abc	input error cleared/rejected	No fare

Reserve boundary: with a balance of Rs 62 and a fare of Rs 12, the remaining balance is exactly Rs 50, so the journey is accepted. With Rs 61.99, the remaining amount is Rs 49.99, so it is rejected.

Five-journey hand calculation example: Inner off-peak Rs 10 + Inner peak Rs 12 + Middle off-peak Rs 20 + Outer peak Rs 36 + Outer off-peak Rs 30 = Rs 108 total fare. This is the gross fare total before applying any card reserve rule; acceptance depends on starting balance and each journey in order.

Reusable test-driven C++ core

```
#include <array>
#include <iostream>
#include <string>

enum class Zone { Inner, Middle, Outer };
constexpr long long fares[] = {1000, 2000, 3000};

bool isPeak(int h, int m) {
    if (h < 0 || h > 23 || m < 0 || m > 59) return false;
    int t = h * 60 + m;
    return (t >= 480 && t < 600) || (t >= 1020 && t < 1200);
}
long long calculateFare(Zone z, int h, int m) {
    int idx = static_cast<int>(z);
    if (idx < 0 || idx >= 3 || h < 0 || h > 23 || m < 0 || m > 59)
        return -1;
    long long f = fares[idx];
    return isPeak(h, m) ? f * 120 / 100 : f;
}
int main() {
    struct Test { const char* name; Zone z; int h,m; long long expected; };
    const std::array<Test,6> tests{{
        {"Inner 07:59",Zone::Inner,7,59,1000},
        {"Inner 08:00",Zone::Inner,8,0,1200},
        {"Middle 10:00",Zone::Middle,10,0,2000},
        {"Outer 17:00",Zone::Outer,17,0,3600},
        {"Outer 20:00",Zone::Outer,20,0,3000},
        {"Outer 19:59",Zone::Outer,19,59,3600}
    }};
    for (const auto& t : tests) {
        long long actual = calculateFare(t.z,t.h,t.m);
        std::cout << t.name << ": expected=" << t.expected
                  << ", actual=" << actual
                  << (actual == t.expected ? " PASS\n" : " FAIL\n");
    }
}
```

The source above demonstrates the test-table approach. Add separate negative tests for invalid enum input, negative recharge, and non-numeric console input in your repository. The test harness should compare actual values to expected values and print PASS/FAIL.

Alternative concept and complexity

A table-driven test loop runs t tests in O(t) time and stores O(t) test records. Adding a boundary case is a new row rather than a new hand-written if statement. This is easier to maintain and gives repeatable regression evidence.

Task 10 – Compare Expectations with Execution (4 marks)

For a transparent workload estimate, assume every journey costs the maximum original peak fare of Rs 36. This is a conservative upper-bound illustration, not a measured real-world traffic mix.

Journeys	Total at Rs 36 each	Total paise	Fits signed 32-bit int paise?
20,000	Rs 720,000	72,000,000	Yes
500,000	Rs 18,000,000	1,800,000,000	Yes, narrowly
3,000,000	Rs 108,000,000	10,800,000,000	No

At 3,000,000 maximum-fare journeys, a 32-bit signed int overflows. long long safely holds this total. double can represent large magnitudes but not every decimal fraction exactly; integer paise avoids currency rounding. Each addition is O(1). Typical storage: int 4 bytes, long long 8 bytes, double 8 bytes on the stated target ABI; confirm with sizeof.

The assignment asks for measured totals and compiler warnings. Record your own evidence using commands such as:

```
g++ --version
g++ -std=c++17 -Wall -Wextra -Wconversion -Wsign-conversion task10.cpp -o task10
./task10
```

Common warnings to discuss if they appear: -Wsign-compare means signed and unsigned values are compared; -Wconversion means an implicit conversion may change a value; -Wunused-variable/-Wunused-parameter means a declared item is unused. Do not claim a warning was produced unless it appears in your compiler output.

Alternative numeric type comparison

Accumulator	3,000,000 × Rs 36	Time/add	Storage (typical)	Assessment
int paise	10,800,000,000 exceeds INT_MAX	O(1)	4 bytes	Overflow; reject for network total.
long long paise	10,800,000,000 fits	O(1)	8 bytes	Recommended exact currency accumulator.
double rupees	108,000,000.0 magnitude fits	O(1)	8 bytes	Range is ample but decimal representation may be approximate.

Task 11 – Recommend and Defend the Final Design (4 marks)

Recommendation: retain the object-oriented architecture, but modify it into a policy-driven design. Use enum class Zone, integer paise (long long), constexpr rule constants, SmartCard with private balance, Journey for audit data, and a FarePolicy/FareTable class for pricing. Validate all input before calculating or deducting.

- Fare revision: update one fare table / policy definition rather than search-and-replace numeric literals.
- Whole-network correctness: long long paise is sufficient for the stated maximum of 3,000,000 journeys under the supplied fare assumptions; use checked addition if the system may grow beyond the analysed maximum.
- Limitations: concession percentage is unspecified; tax/rounding policy is not provided; refunds, transfers, fare caps, multi-leg journeys, concurrency and transaction persistence are outside the initial scope.

- Evidence: use the Task 9 test matrix and Task 10 workload totals. Insert your actual execution screenshots, compiler warnings and timing measurements before submission.

Alternative: fare configuration file

A configuration file allows fare revisions without recompiling the program. Startup must read and validate z fare entries, which takes O(z) time and O(z) storage; each later lookup remains O(1) with an array/map appropriate to the zone key. It introduces operational risks: missing file, malformed values, invalid negative fares, partial updates and version control. A robust system validates the complete file and falls back safely or refuses to open gates. Recompiling has deployment cost, while configuration loading has startup and configuration-management cost.

Task 12 – Re-evaluate and Adapt the Design (Day-4 Design Challenge, 8 marks)

A. Impact analysis (2 marks)

Original element	Change required
Zone enum	Add Zone::Airport; ensure enum values map to the fare table.
Fare table	Add Airport base fare = 6000 paise (Rs 60).
Peak expression	Peak rule remains unchanged: [08:00,10:00) and [17:00,20:00).
Weekend rule	Add date/day input and a separate 10% discount policy; weekend discount must not combine with peak surcharge.
Switch logic	Add Airport case, or replace switch with validated indexed fare table.
SmartCard	Can remain; it still validates the post-payment reserve and stores balance privately.
New Pass class	Add two selected zones and active/expiry status; matching-zone travel costs zero under the stated pass assumption.

B. Evaluate original design (2 marks)

The original switch can absorb Airport by adding a case, but each new zone requires a code change and risks missing a branch. A table indexed by Zone is easier to extend by adding one entry, provided index validation is strict. SmartCard remains suitable because balance and reserve behaviour do not depend on how fare is calculated; it should be retained. Fare calculation should be modified because peak and weekend rules now interact.

Worked examples (before any concession):

Journey	Rule	Fare
Airport weekday off-peak	Rs 60 base	Rs 60.00
Airport weekday peak	Rs 60 × 1.20	Rs 72.00
Airport weekend peak	Weekend 10% discount; no peak surcharge	Rs 54.00
Outer weekend off-peak	Rs 30 × 0.90	Rs 27.00
Middle weekday peak	Rs 20 × 1.20	Rs 24.00

C. Revised design and implementation (2 marks)

The following implementation uses a fare array, an explicit weekend flag, and a MonthlyPass covering exactly two chosen zones. Assumption: an active pass makes matching-zone journeys free; other journeys use normal fare rules. The weekend discount applies to the base fare and suppresses peak surcharge.

```
#include <array>
#include <iostream>
#include <optional>
```

```
enum class Zone { Inner, Middle, Outer, Airport };
constexpr std::array<long long,4> BASE_FARE{
    1000, 2000, 3000, 6000
};
constexpr long long MIN_RESERVE = 5000;

class MonthlyPass {
    Zone first_, second_;
    bool active_;
public:
    MonthlyPass(Zone a, Zone b, bool active)
        : first_(a), second_(b), active_(active) {}
    bool covers(Zone z) const {
        return active_ && (z == first_ || z == second_);
    }
};

class SmartCard {
    long long balance_;
public:
    explicit SmartCard(long long initial) : balance_(initial) {}
    long long balance() const { return balance_; }
    bool canTravel(long long fare) const {
        return fare >= 0 && balance_ >= fare &&
               balance_ - fare >= MIN_RESERVE;
    }
    bool deduct(long long fare) {
        if (!canTravel(fare)) return false;
        balance_ -= fare;
        return true;
    }
};

long long calculateFare(Zone z, int hour, int minute,
                        bool weekend, const MonthlyPass* pass = nullptr) {
    int index = static_cast<int>(z);
    if (index < 0 || index >= static_cast<int>(BASE_FARE.size()) ||
        hour < 0 || hour > 23 || minute < 0 || minute > 59)
        return -1;

    if (pass != nullptr && pass->covers(z)) return 0;

    long long fare = BASE_FARE[static_cast<std::size_t>(index)];
    if (weekend) {
        fare = fare * 90 / 100; // no peak surcharge on weekends
    } else {
        int t = hour * 60 + minute;
        bool peak = (t >= 480 && t < 600) ||
                    (t >= 1020 && t < 1200);
        if (peak) fare = fare * 120 / 100;
    }
    return fare;
}

int main() {
    MonthlyPass pass(Zone::Inner, Zone::Middle, true);
    SmartCard card(20000); // Rs 200
    const long long f1 = calculateFare(Zone::Airport, 18, 0, true, &pass);
    const long long f2 = calculateFare(Zone::Airport, 18, 0, false, &pass);
    const long long f3 = calculateFare(Zone::Inner, 9, 0, false, &pass);

    std::cout << "Airport weekend peak: " << f1 << " paise\n";
```

```
    std::cout << "Airport weekday peak: " << f2 << " paise\n";
    std::cout << "Inner covered by pass: " << f3 << " paise\n";

    if (card.deduct(f1)) std::cout << "Weekend Airport accepted\n";
    else std::cout << "Weekend Airport denied\n";
}
```

Expected output for this demonstration:

```
Airport weekend peak: 5400 paise
Airport weekday peak: 7200 paise
Inner covered by pass: 0 paise
Weekend Airport accepted
```

D. Evidence-based defence (2 marks)

- Retain SmartCard: its responsibility (stored value and reserve enforcement) is unchanged by new fare rules.
- Modify fare calculation: separate the weekend policy from peak calculation so the two discounts cannot accidentally combine.
- Replace or centralize the switch in a validated four-entry fare table: adding Airport becomes a data change, not a new repeated branch.
- Add MonthlyPass as a separate object: pass eligibility is not a property of the card's cash balance and should be checked before cash fare deduction.
- Run regression tests for all four zones, weekday peak/off-peak, weekend peak/off-peak, pass-covered and non-covered zones, expired/inactive pass, and the Rs 50 reserve boundary.

Trade-off: the revised design is more extensible and separates responsibilities, but it adds classes and validation paths. For a small fixed three-zone system the original switch was adequate; with repeated additions and interacting discounts/passes, the table + policy + pass design is easier to maintain. Record actual test results on your own compiler before claiming empirical validation.

GitHub repository and execution checklist

9. Create repository named DSA01-<RegisterNumber>-Assignment1.
10. Create Task01 through Task12 folders. Keep source code, test input files and actual output screenshots in the matching folder.
11. Compile each program with: g++ -std=c++17 -Wall -Wextra -Wconversion -pedantic filename.cpp -o program
12. Run the program and save terminal output screenshots. For interactive programs, include valid and invalid input screenshots.
13. Add README.md containing student name, register number, compiler/version, exact compile commands, run instructions and a short result for every task.
14. Commit progressively: at least one meaningful commit per task. Do not upload all work in one final commit.
15. Share the repository with faculty and paste the working GitHub URL on the cover page and submission form.

Suggested README.md template

```
# DSA01 - Assignment 1: Namma Metro Smart-Card Fare System

- Name: <Your Name>
- Register Number: <Your Register Number>
- Compiler: g++ <your version>
- Standard: C++17

## Compile
g++ -std=c++17 -Wall -Wextra -Wconversion -pedantic main.cpp -o main

## Run
./main
# Windows:
main.exe
```

```
## Task results
- Task01: OOP analysis and object model.
- Task02: Typed constants and enum class.
- Task03: Integer-paise data representation.
- Task04: Conversion prediction and validation.
- Task05: Operator precedence prediction.
- Task06: Switch-based fare calculation.
- Task07: Procedural and SmartCard implementations.
- Task08: Safe input and formatted output.
- Task09: Fare test matrix and boundary cases.
- Task10: Scale and numeric type comparison.
- Task11: Final architecture recommendation.
- Task12: Fourth zone, weekend discount and monthly pass.

## Evidence
Each Task folder contains source files, inputs (if any), and screenshots of actual execution.
```

Final submission reminders

- Fill the student details and replace the GitHub placeholder with your real repository URL.
- Run the two prediction programs only after writing your prediction in your notebook/report; then compare with the displayed expected result.
- Confirm sizeof values and compiler warnings on your installed g++ version; ABI and compiler options can affect diagnostics.
- Add the actual measured runtime/rounding experiment requested in Tasks 3 and 10. The report provides the method and expected mathematical totals, not fabricated laptop measurements.
- Concession percentage and detailed eligibility were not specified in the assignment statement. Ask faculty for the required percentage and whether it combines with peak/weekend discounts before implementing that rule.